SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA0509

COURSE OUTCOME COVERED

CO3: *Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.* (BL2, BL3)

Activity Tool: Experiments (High)
Assessment Tool Weightage: 40%

QUESTIONS			Total Marks: 50
Q.No	**Question**	**Bloom's Level**	**Marks**
1	For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys.	BL2 – Understand	**5**
2	Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.	BL3 – Apply	**5**
3	Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.	BL3 – Apply	**5**
4	Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.	BL3 – Apply	**5**
5	Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.	BL3 – Apply	**5**
6	Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.	BL3 – Apply	**5**

7	Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.	BL3 – Apply	**5**
8	Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).	BL3 – Apply	**5**
9	Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.	BL2 – Understand	**5**
10	Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.	BL3 – Apply	**5**

Code of Conduct:

I A Mohammad Naheed (192525336) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: A Mohammad Naheed

1. For the relation STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade), identify all functional dependencies and determine the candidate keys

Given the relation:

STUDENT_COURSE(SID, SName, CID, CName, Instructor, Grade)

Understand the attributes

Attribute	Meaning
SID	Student ID
SName	Student Name
CID	Course ID
CName	Course Name
Instructor	Course Instructor
Grade	Grade obtained by student

Assume that:

- Each SID identifies exactly one student.
- Each CID identifies exactly one course and instructor.
- A student's grade is determined by the combination of student and course.

Identify Functional Dependencies

FD 1: Student ID determines Student Name

A particular student ID belongs to only one student.

$$SID \rightarrow SName$$

Example:
SID = S101 → SName = Ravi

FD 2: Course ID determines Course Name

Each course has a unique course ID.

$$CID \rightarrow CName$$

Example:
CID = C01 → CName = DBMS

FD 3: Course ID determines Instructor

Assuming each course has one instructor:

$$CID \rightarrow Instructor$$

Example:
CID = C01 → Instructor = Kumar

Therefore, we can combine the two:

$$CID \rightarrow CName, Instructor$$

FD 4: Student ID + Course ID determines Grade

A student's grade depends on **which course** the student took.
Therefore:

$$SID,CID \rightarrow Grade$$

For example:
SID = S101 + CID = C01 → Grade = A

List All Main Functional Dependencies

Therefore, the functional dependencies are:

SID→SName
CID→CName
CID→Instructor
SID,CID→Grade

Or in combined form:

SID→SName,CID→CName,Instructor,SID,CID→Grade

Find the Candidate Key

We need an attribute set that can determine **all attributes** of the relation.

Try:

{SID,CID}

Calculate its closure:

(SID,CID)+

Start with:

(SID,CID)+={SID,CID}

Apply FD 1

We know:

SID→SName

Therefore add SName.

(SID,CID)+={SID,CID,SName}

Apply FD 2 and FD 3

We know:

CID→CName

And

CID→Instructor

Therefore add CName and Instructor.

(SID,CID)+={SID,CID,SName,CName,Instructor}

Apply FD 4

We know:

SID,CID→Grade

Therefore add Grade.

(SID,CID)+={SID,SName,CID,CName,Instructor,Grade}

We have obtained **all attributes** of the relation.

Therefore:

(SID,CID) is a superkey

Check Minimality

Now check whether we can remove either attribute.

Remove SID

Try:

CID+

We get:

CID+={CID,CName,Instructor}

It cannot determine:

- SID
- SName
- Grade

Therefore, CID alone is **not a key**.

Remove CID
Try:
SID+
We get:
SID+={SID,SName}
It cannot determine:

- CID
- CName
- Instructor
- Grade

Therefore, SID alone is **not a key**.

Since neither SID nor CID can be removed:
(SID,CID) is a minimal superkey
Hence it is a **candidate key**.

Functional Dependencies
SID→SName
CID→CName
CID→Instructor
SID,CID→Grade
Candidate Key
Candidate Key=(SID,CID)

2. Demonstrate the process of converting an unnormalized relation into 1NF with step-by-step justification and example.

ANS:

Understand 1NF
A relation is in First Normal Form (1NF) when:

1. Every attribute contains atomic (single) values.
2. There are no repeating groups.
3. Each cell contains only one value.

Consider an Unnormalized Relation
Consider a student-course relation:
STUDENT_COURSE

SID	SName	Courses	Grades
S101	Riya	DBMS, Java	A, B
S102	Priya	Python, Cloud	A, A
S103	Haroon	DBMS	B

Why is this UNF?
The Courses and Grades attributes contain multiple values in a single cell.
For example:
Ravi → DBMS, Java
and:
Ravi → A, B
Therefore, the values are not atomic.
So:
STUDENT_COURSE is not in 1NF

Identify the Repeating Groups
For student S101:
Courses = DBMS, Java
Grades = A, B
There are multiple course and grade values.

For student S102:
Courses = Python, Cloud
Grades = A, A
Again, there are repeating values.
These repeating groups must be removed.

Separate the Repeating Values
Each course should be placed in a separate row, along with its corresponding grade.

For Ravi:

SID	SName	Course	Grade
S101	Riya	DBMS	A
S101	Riya	Java	B

For Priya:

SID	SName	Course	Grade
S102	Priya	Python	A
S102	Priya	Cloud	A

For Arun:

SID	SName	Course	Grade
S103	Haroon	DBMS	B

Form the 1NF Relation
Combine all the rows:
STUDENT_COURSE_1NF

SID	SName	Course	Grade
S101	Riya	DBMS	A
S101	Riya	Java	B
S102	Priya	Python	A
S102	Priya	Cloud	A
S103	Haroon	DBMS	B

Verify Atomicity
Now check each cell.
SID
S101
S101
S102
S102
S103
Each cell contains one value.

SName
Riya
Riya
Priya
Priya
Haro
on
Each cell contains one value.

Course
DBMS
Java
Python
Cloud
DBMS
Each cell contains one value.
Grade
A
B
A
A
B
Each cell contains one value.
Therefore, all attributes contain atomic values.

Determine the Key
A student can take multiple courses, so SID alone cannot uniquely identify a row.
For example:
S101 → DBMS
S101 → Java
Therefore, we use:
(SID,Course)
as the candidate key for this simplified relation.
It uniquely identifies each student-course record**.**

Compare UNF and 1NF

UNF	**1NF**
Multiple values in a cell	One value per cell
Repeating groups exist	Repeating groups removed
Not atomic	Atomic
Difficult to query	Easier to query
Example: DBMS, Java	DBMS and Java in separate rows

Why Conversion to 1NF is Necessary
Converting the relation to 1NF provides:

- Atomic values
- No repeating groups
- Easier searching and querying
- Better organization of data
- A proper foundation for further normalization into 2NF, 3NF, etc.

Conversion Flow

UNNORMALIZED RELATION
↓
Identify repeating groups
↓
Separate multiple values
↓

Create separate rows
↓
Ensure every cell contains one value
↓
Identify suitable key
↓
1NF

The unnormalized relation contains repeating/multivalued attributes, such as multiple courses and grades in a single cell. To convert it into 1NF, each multivalued attribute is separated into individual rows so that every cell contains exactly one atomic value. The resulting relation satisfies the requirements of 1NF, and (SID, Course) can be used as the candidate key for the example.

UNF→Remove Repeating Groups→Atomic Values→1NF

3. Given relation R(A,B,C,D,E) with FDs: A→BC, BC→D, D→E. Analyze and decompose it to 2NF eliminating partial dependencies.

Write the Relation

R(A,B,C,D,E)

So the attributes are:

Attribute

A

B

C

D

E

Functional Dependencies

Given:

FD1

A→BC

This means A determines B and C.

FD2

BC→D

This means B and C together determine D.

FD3

D→E

This means D determines E.

Candidate Key

To determine the candidate key, calculate the closure of A.
A+
Initially:

A+={A}

Apply

Since A determines B and C:

A+={A,B,C}

Apply

We now have both B and C.
Therefore:

BC→D

So:

A+={A,B,C,D}

Apply
We have D, so:

D→E

Therefore:

A+={A,B,C,D,E}

The closure contains all attributes of R.

Therefore:

A is a candidate key

Check Whether Partial Dependency Exists

Definition

A partial dependency occurs when a non-key attribute depends on part of a composite candidate key.

For example, if the candidate key were:

(A,B)

and we had:

A→C

then C would depend only on part of the key A.

That would be a partial dependency.

In our relation

The candidate key is:

A

It contains only one attribute.

Therefore, there is no "part of the key" on which a non-key attribute can partially depend.

Hence:

No Partial Dependency Exists

Determine 2NF

A relation is in 2NF if:

1. It is in 1NF.
2. It has no partial dependency of a non-prime attribute on a candidate key.

Here:

- Candidate key = A
- A is a single-attribute key.
- Therefore, partial dependency is impossible.

Hence:

R(A,B,C,D,E) is already in 2NF

Is Decomposition Required?

No decomposition is required for 2NF.

This is an important point.

The given FDs contain:

A→BC

BC→D

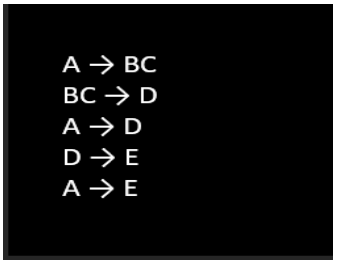
D→E

There are transitive dependencies, but transitive dependencies are a 3NF issue, not a 2NF issue.

For example:

And

means:



So there are transitive dependencies such as:

$$A \rightarrow D \rightarrow E$$

But these do not violate 2NF.

Final 2NF Relation

Therefore, the relation remains:

$$R(A,B,C,D,E,F)$$

No decomposition is needed to achieve 2NF.

Final Answer

Given:

$$R(A,B,C,D,E,F)$$

$$A \rightarrow BC,\ BC \rightarrow D,\ D \rightarrow E$$

Candidate Key:

$$A$$

because:

$$A^+ = (A,B,C,D,E)$$

Partial Dependency:

None

because the candidate key contains only one attribute.

2NF Status:

R is already in 2NF

Decomposition:

No Decomposition is required for 2NF

Observation:

There are transitive dependencies, so further decomposition may be required when converting to 3NF.

4. Apply 3NF decomposition on the relation EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID) with transitive dependencies.

Identify the Attributes

Attribute	Meaning
EmpID	Employee ID
EmpName	Employee Name
DeptID	Department ID
DeptName	Department Name
ManagerID	Manager ID

Identify Functional Dependencies

Assume each employee has one employee name, department, and manager.
Therefore:

EmpID→EmpName,DeptID,ManagerID

Also, each department ID determines its department name:

DeptID→DeptName

Find the Candidate Key

Calculate:

EmpID+

Initially:

EmpID+={EmpID}

Using:

EmpID→EmpName,DeptID,ManagerID

we get:

EmpID+={EmpID,EmpName,DeptID,ManagerID}

Now use:

DeptID→DeptName

Therefore:

EmpID+={EmpID,EmpName,DeptID,DeptName,ManagerID}

Since the closure contains **all attributes**, EmpID is a candidate key.

Candidate Key=EmpID

Identify the Transitive Dependency

We have:

EmpID→DeptID

and:

DeptID→DeptName

Therefore:

EmpID→DeptID→DeptName

So, DeptName depends on EmpID **indirectly through DeptID**.
This is called a **transitive dependency**.

Check 3NF Condition

A relation is in **3NF** if, for every functional dependency:
$X \rightarrow Y$
either:

1. X is a superkey, **or**
2. Y is a prime attribute.

Here:
DeptID→DeptName
But DeptID is **not a superkey** of the original Employee relation, and DeptName is not a prime attribute.
Therefore:
The relation violates 3NF

Decompose the Relation
To remove the transitive dependency, separate the department information.

Relation 1: EMPLOYEE
EMPLOYEE(EmpID,EmpName,DeptID,ManagerID)
Functional dependency:
EmpID→EmpName,DeptID,ManagerID
Here, EmpID is the primary key.

Relation 2: DEPARTMENT
DEPARTMENT(DeptID,DeptName)
Functional dependency:
DeptID→DeptName
Here, DeptID is the primary key.

Final 3NF Schema
EMPLOYEE

EmpID ← Primary Key
EmpName
DeptID ← Foreign Key
ManagerID

DEPARTMENT

DeptID ← Primary Key
DeptName
Relationship:

DEPARTMENT
|
| 1
|
| M
↓
EMPLOYEE
One department can have **many employees**, while each employee belongs to one department.

Why This is 3NF

EMPLOYEE

EmpID→EmpName,DeptID,ManagerID

EmpID is the key, so the dependency satisfies 3NF.

DEPARTMENT

DeptID→DeptName

DeptID is the key, so the dependency satisfies 3NF.

Therefore:

EMPLOYEE and DEPARTMENT are in 3NF

Advantages of Decomposition

Before decomposition

The original table might contain:

EmpID	EmpName	DeptID	DeptName	ManagerID
E01	Riya	D01	CSE	M01
E02	Priya	D01	CSE	M01
E03	Haroon	D02	ECE	M02

CSE is repeated for every employee in department D01.

This can cause:

- **Update anomaly** – changing the department name requires multiple updates.
- **Insertion anomaly** – a department cannot easily be stored without an employee.
- **Deletion anomaly** – deleting the last employee may remove department information.

After decomposition, department information is stored only once.

Original Relation

EMPLOYEE(EmpID,EmpName,DeptID,DeptName,ManagerID)

Functional Dependencies

EmpID→EmpName,DeptID,ManagerID DeptID→DeptName

Transitive Dependency

EmpID→DeptID→DeptName

Candidate Key

EmpID

3NF Decomposition

EMPLOYEE(EmpID,EmpName,DeptID,ManagerID) DEPARTMENT(DeptID,DeptName)

Conclusion

The original relation violates **3NF** because DeptName has a transitive dependency on the candidate key EmpID through DeptID. By decomposing it into **EMPLOYEE** and **DEPARTMENT**, the transitive dependency is removed, redundancy is reduced, and the resulting relations satisfy **3NF**.

5. Verify whether R(A,B,C,D) with FDs: AB→C, C→D, D→A is in BCNF. If not, decompose it into BCNF.

Check whether R(A,B,C,D) is in BCNF. If not,decompose it.

Given Relation

[
R(A,B,C,D)
]

Functional Dependencies:

[
AB\rightarrow C
]

[
C\rightarrow D
]

[
D\rightarrow A
]

Find Candidate Keys

Find (AB^+)

[
AB^+={A,B}
]

Using:

[
AB\rightarrow C
]

[AB^+={A,B,C}
]

Using:

[
C\rightarrow D
]

[
\boxed{AB^+={A,B,C,D}}
]

So, **AB is a candidate key**.

Similarly:

[
BC^+={A,B,C,D}
]

So **BC is also a candidate key**.

And:

[
BD^+={A,B,C,D}
]

So **BD is also a candidate key**.

Therefore:

[
\boxed{Candidate\ Keys=AB,\ BC,\ BD}
]

Check BCNF Condition

BCNF Rule

For every functional dependency:

[
X\rightarrow Y
]

X must be a superkey.

Check the given FDs:

FD	Is determinant a superkey?	Result
(AB\rightarrow C)	Yes	✓
(C\rightarrow D)	No	✗
(D\rightarrow A)	No	✗

Therefore:

[
\boxed{R\text{ is NOT in BCNF}}
]

Decompose the Relation

Take the violating FD:

[
C\rightarrow D
]

Create the first relation:

[
\boxed{R_1(C,D)}
]

The remaining attributes form:

[
\boxed{R_2(A,B,C)}
]

Therefore:

[
\boxed{R(A,B,C,D)\rightarrow R_1(C,D)+R_2(A,B,C)}
]

Check (R_1(C,D))

FD:

[
C\rightarrow D
]

Here, C determines all attributes of (R_1).

Therefore, C is a key.

[
\boxed{R_1(C,D)\text{ is in BCNF}}
]

Check (R_2(A,B,C))

FD:

[
AB\rightarrow C
]

Here, AB is a key of (R_2).

Therefore:

[

\boxed{R_2(A,B,C)\text{ is in BCNF}}
]

Final Answer

[
\boxed{R_1(C,D)}
]
[
\boxed{R_2(A,B,C)}
]

Thus, the original relation **R is not in BCNF**, because (C\rightarrow D) and (D\rightarrow A) violate the BCNF condition. After decomposition, both relations are in **BCNF**.

6. Experimentally verify lossless-join decomposition for a given relation using the tabular (Chase) algorithm.

ANS:

Given:

Relation:

R(A,B,C)

Functional Dependency:

$A \rightarrow B$

Decomposition:

R1(A,B),R2(A,C)

We need to verify whether the decomposition is **lossless-join** using the **Chase Algorithm**.

Create the Chase Table

Write all attributes of the original relation as columns.

Relation	**A**	**B**	**C**
R1(A,B)	a	b	c_1
R2(A,C)	a	b_2	c

Here:

- a, b, c are distinguished symbols.
- b2, c1 are different symbols.

Apply the Functional Dependency

Given:

$A \rightarrow B$

This means:

If two rows have the same value of A, their B values must also be the same.

Compare the A Column

Both rows have:

$A=a$

Therefore, according to:

$A \rightarrow B$

the B values must be equal.

Currently:

$b1=b2$

So change:

$b2 \rightarrow b$

Update the Chase Table

Relation	A	B	C
R1	a	b	c_1
R2	a	b	c

Check the Chase Result

The Chase process has successfully enforced the functional dependency:

$A \rightarrow B$

The common values are propagated correctly between the decomposed relations.

Therefore, the decomposition does not lose information.

Decomposition is Lossless-Join

Final Conclusion

The decomposition:

$R(A,B,C) \rightarrow R1(A,B), R2(A,C)$

is **lossless** because the Chase algorithm succeeds in satisfying the functional dependency $A \rightarrow B$.

7. Identify the multi-valued dependencies in TEACHER(TID, Subject, Hobby) and decompose it into 4NF.

ANS:

Multivalued Dependencies and 4NF

Given relation:

TEACHER(TID,Subject,Hobby)

We need to **identify the multivalued dependencies (MVDs)** and **decompose the relation into 4NF**.

Understand the Relation

A teacher can have:

- Multiple subjects
- Multiple hobbies

For example:

TID	Subject	Hobby
T01	DBMS	Cricket
T01	DBMS	Music
T01	Java	Cricket
T01	Java	Music

Here, the **subjects and hobbies are independent** of each other.

Identify Multivalued Dependencies

Since one teacher can have many subjects:

TID→Subject

Since one teacher can have many hobbies:

TID→Hobby

These are the **multivalued dependencies**.

Check 4NF

4NF Rule

A relation is in **4NF** if, for every non-trivial MVD:

X→Y

X must be a **superkey**.

Here:

TID→Subject

But TID alone does not determine one subject because a teacher can teach multiple subjects.

Similarly:

TID→Hobby

Therefore, the original relation **violates 4NF**.

TEACHER is NOT in 4NF

Decompose the Relation

Separate the independent multivalued attributes.

Relation 1: Teacher-Subject

TEACHER_SUBJECT(TID,Subject)

Example:

TID	Subject
T01	DBMS
T01	Java

Relation 2: Teacher-Hobby

TEACHER_HOBBY(TID,Hobby)

Example:

TID	Hobby
T01	Cricket
T01	Music

Check 4NF

TEACHER_SUBJECT

TID→Subject

TID is the key for this relation.

Therefore, it satisfies 4NF.

TEACHER_HOBBY

TID→Hobby

TID is the key for this relation.

Therefore, it satisfies 4NF.

Multivalued Dependencies:

TID→Subject TID→Hobby

4NF Decomposition:

TEACHER_SUBJECT(TID,Subject) TEACHER_HOBBY(TID,Hobby)

Conclusion:

The original relation has independent multivalued attributes, causing a **4NF violation**. Decomposing it into TEACHER_SUBJECT and TEACHER_HOBBY removes the redundancy and gives relations in **4NF**.

8. Demonstrate how join dependencies lead to redundancy. Decompose a given relation to 5NF (Project-Join Normal Form).

Given relation

Consider:
SPJ(Supplier,Part,Project)
We need to show how a **join dependency causes redundancy** and decompose the relation into **5NF (Project-Join Normal Form)**.

Understand the Relation
Suppose a supplier can:

- Supply a particular part.
- Work on a particular project.
- A particular part can be used in a particular project.

Example:

Supplier	Part	Project
S1	P1	J1
S1	P2	J1
S2	P1	J1

Identify the Redundancy
The three attributes contain separate relationships:
Supplier−Part Supplier−Project Part−Project
If these relationships are stored together in one table, combinations may be repeated.
For example, information about:
S1 supplies P1
S1 works on J1
P1 is used in J1
may be stored repeatedly as part of different combinations.
Therefore:
Join dependency can cause redundancy

Identify the Join Dependency
The original relation can be represented using three smaller relations:
SP(Supplier,Part) SJ(Supplier,Project) PJ(Part,Project)
The original relation can then be reconstructed by joining them:
SPJ=SP⋈SJ⋈PJ
This is the **join dependency**.

Decompose the Relation
Relation 1
SP(Supplier,Part)
Relation 2
SJ(Supplier,Project)
Relation 3

PJ(Part,Project)

Why This Gives 5NF

5NF requires that every non-trivial **join dependency** is implied by the candidate keys.
The decomposition separates the independent relationships and avoids unnecessary redundancy.
Thus:

SPJ→SP, SJ, PJ

and the original relation can be reconstructed by joining the three relations.

Original relation:

SPJ(Supplier,Part,Project)

Join dependency:

SPJ=SP⋈SJ⋈PJ

5NF Decomposition:

SP(Supplier,Part) SJ(Supplier,Project) PJ(Part,Project)

Conclusion:

The original relation may contain **redundant combinations** because three independent relationships are stored together. Decomposing it into SP, SJ, and PJ removes the redundancy and gives the required **5NF (Project-Join Normal Form)** structure.

9. Given a poorly designed database schema for a college, identify all anomalies (insertion, deletion, update) and normalize it to 3NF.

Consider the poorly designed relation:
COLLEGE(StudentID,StudentName,CourseID,CourseName,Instructor,DeptID,DeptName)

Identify Functional Dependencies
Assume:
StudentID→StudentName CourseID→CourseName,Instructor,DeptID DeptID→DeptName
So there is a transitive dependency:
CourseID→DeptID→DeptName

Identify Anomalies
1. Insertion Anomaly
Suppose a new course is created but **no student has enrolled yet**.
We cannot easily insert the course information because StudentID would be required.
Cannot insert course independently
2. Deletion Anomaly
Suppose the only student enrolled in a course is deleted.
The course and instructor information may also be lost.
Deleting a student may delete course information
3. Update Anomaly
Suppose the department name changes:
CSE→Computer Science
The department name may occur in many rows, so every row must be updated.
If one row is missed, inconsistent data occurs.
Same information must be updated in multiple rows

Normalize to 3NF
Separate the information into different relations.
STUDENT
STUDENT(StudentID,StudentName)
Primary Key:
StudentID
DEPARTMENT
DEPARTMENT(DeptID,DeptName) Primary Key:
DeptID
COURSE
COURSE(CourseID,CourseName,Instructor,DeptID)
Primary Key:
CourseID
DeptID is a foreign key.
ENROLLMENT
Because a student can take many courses:
ENROLLMENT(StudentID,CourseID)
Primary Key:
(StudentID,CourseID)
Both are also foreign keys.

Final 3NF Schema
STUDENT(StudentID,StudentName) DEPARTMENT(DeptID,DeptName)
COURSE(CourseID,CourseName,Instructor,DeptID) ENROLLMENT(StudentID,CourseID)

Why It is 3NF
The transitive dependency:

CourseID→DeptID→DeptName

has been removed by creating a separate DEPARTMENT table.
Therefore, the relations are in **3NF**.

1. Write the poorly designed table
COLLEGE(StudentID,StudentName,CourseID,CourseName,Instructor,DeptID,DeptName)
2. Identify anomalies

- Insertion anomaly
- Deletion anomaly
- Update anomaly

3. Identify transitive dependency

CourseID→DeptID→DeptName

4. Decompose
STUDENT(StudentID,StudentName) COURSE(CourseID,CourseName,Instructor,DeptID) DEPARTMENT(DeptID,DeptName) ENROLLMENT(StudentID,CourseID)
5. Conclusion: The decomposition reduces redundancy and removes insertion, deletion, and update anomalies, producing a **3NF design**.

Q10. Compute the closure of attribute sets for the given FDs and determine all candidate keys for the relation.

Relation:

R(A, B, C, D, E)

Functional Dependencies (FDs):

- $A \rightarrow BC$
- $BC \rightarrow D$
- $D \rightarrow E$
-

1. Find A^+

Start with:
$A^+ = \{A\}$
Using $\mathbf{A \rightarrow BC}$, add B and C:
$A^+ = \{A, B, C\}$
Using $\mathbf{BC \rightarrow D}$, add D:
$A^+ = \{A, B, C, D\}$
Using $\mathbf{D \rightarrow E}$, add E:
$A^+ = \{A, B, C, D, E\}$
Since A^+ contains all attributes of R, **A is a key**.

2. Find B^+
Start with:
$B^+ = \{B\}$
No functional dependency can be applied.
$B^+ = \{B\}$
Since B^+ does not contain all attributes, **B is not a key**.

3. Find C^+
Start with:
$C^+ = \{C\}$
No functional dependency can be applied.
$C^+ = \{C\}$
Since C^+ does not contain all attributes, **C is not a key**.

4. Find D^+
Start with:
$D^+ = \{D\}$
Using $\mathbf{D \rightarrow E}$, add E:
$D^+ = \{D, E\}$
Since D^+ does not contain all attributes, **D is not a key**.

5. Find E^+
Start with:
$E^+ = \{E\}$
No functional dependency can be applied.
$E^+ = \{E\}$
Since E^+ does not contain all attributes, **E is not a key**.

Candidate Key
Only $\mathbf{A^+}$ contains all the attributes of the relation:
$A^+ = \{A, B, C, D, E\}$
Therefore, **A is the candidate key**.

Final Answer

Attribute Set	Closure	Key?
$\mathbf{A^+}$	{A, B, C, D, E}	**Yes**
$\mathbf{B^+}$	{B}	No
$\mathbf{C^+}$	{C}	No
$\mathbf{D^+}$	{D, E}	No
$\mathbf{E^+}$	{E}	No